

Arquiteturas de Software com Java

Introdução

A arquitetura de software define a estrutura fundamental de um sistema: seus componentes, as relações entre eles e os princípios que guiam seu design e evolução. A escolha arquitetural impacta diretamente a manutenibilidade, escalabilidade, testabilidade e o custo de evolução de qualquer aplicação. Este material apresenta os principais fundamentos conceituais e exemplos práticos implementados em Java.

Fundamentos Conceituais

1. Separação de Responsabilidades (Separation of Concerns)

Separation of Concerns (SoC) é o princípio segundo o qual cada módulo ou camada do sistema deve ter uma responsabilidade única e bem definida, sem interferir nas responsabilidades de outros módulos. O conceito foi formalizado por Edsger Dijkstra em 1974 e constitui a base de praticamente todas as arquiteturas modernas.

A aplicação desse princípio resulta em:

- **Baixo acoplamento:** módulos dependem minimamente uns dos outros, o que facilita alterações isoladas.
- **Alta coesão:** elementos internos de um módulo estão fortemente relacionados entre si e com seu propósito.
- **Testabilidade:** cada unidade pode ser testada de forma independente.

Na prática, a SoC se manifesta na divisão do sistema em camadas horizontais (apresentação, lógica de negócio, acesso a dados) ou verticais (domínios funcionais independentes), dependendo do estilo arquitetural adotado.

2. Inversão de Dependência e o Papel das Abstrações

O **Princípio da Inversão de Dependência (DIP)**, formulado por Robert C. Martin como parte dos princípios SOLID, estabelece duas diretrizes fundamentais:

1. Módulos de alto nível não devem depender de módulos de baixo nível; ambos devem depender de abstrações.
2. Abstrações não devem depender de detalhes; detalhes devem depender de abstrações.

Esse princípio é a base das **Arquiteturas Limpas** (Clean Architecture, Hexagonal Architecture). Ao inverter o fluxo de dependência, o núcleo do domínio permanece independente de frameworks, bancos de dados e interfaces externas. O resultado é um sistema cujo comportamento central pode ser verificado sem que qualquer infraestrutura esteja ativa.

A inversão é implementada por meio de **interfaces** (contratos) que o domínio define e que as camadas externas implementam — padrão conhecido como **Ports and Adapters**.

3. Estilos Arquiteturais: Monolito, SOA e Microserviços

Os estilos arquiteturais descrevem como os componentes de um sistema são organizados e se comunicam. Os três estilos mais relevantes para sistemas corporativos são:

Monolito Todos os módulos do sistema são compilados e implantados como uma única unidade. É o estilo de menor complexidade operacional e adequado para sistemas de pequeno a médio porte ou equipes reduzidas. O principal risco é o acoplamento progressivo entre módulos à medida que o sistema cresce.

Arquitetura Orientada a Serviços (SOA) O sistema é dividido em serviços de granularidade média, que se comunicam por meio de um barramento de mensagens centralizado (ESB). Promove reuso de serviços corporativos, mas o ESB tende a se tornar um gargalo e ponto único de falha.

Microserviços O sistema é decomposto em serviços pequenos e autônomos, cada um com seu próprio processo, banco de dados e ciclo de implantação. A comunicação se dá por APIs (REST, gRPC) ou mensageria assíncrona (Kafka, RabbitMQ). Maximiza a escalabilidade e autonomia das equipes, mas introduz complexidade operacional significativa: rastreamento distribuído, consistência eventual e orquestração de contêineres.

A escolha entre esses estilos deve considerar o tamanho da equipe, o volume de tráfego esperado e a maturidade do time em práticas DevOps.

Exemplos Práticos em Java

Exemplo 1 — Arquitetura em Camadas (Layered Architecture)

A arquitetura em camadas é o padrão mais difundido em aplicações Java empresariais. Divide o sistema em quatro camadas principais: **Apresentação**, **Aplicação (Serviços)**, **Domínio** e **Infraestrutura**. Cada camada só se comunica com a imediatamente abaixo.

O exemplo a seguir modela um sistema simples de pedidos:

```
java

// -----
// CAMADA DE DOMÍNIO
// Entidade central, sem dependências externas
// -----

package com.exemplo.dominio;

public class Pedido {

    private final Long id;
    private final String cliente;
    private double total;

    public Pedido(Long id, String cliente) {
        this.id = id;
        this.cliente = cliente;
        this.total = 0.0;
    }

    public void adicionarItem(double valor) {
        if (valor <= 0) {
            throw new IllegalArgumentException("Valor do item deve ser positivo");
        }
        this.total += valor;
    }

    public Long getId() { return id; }
    public String getCliente(){ return cliente; }
    public double getTotal() { return total; }
}
```

```
java
```

```
// -----
// CAMADA DE INFRAESTRUTURA
// Simula um repositório (acesso a dados)
// -----
package com.exemplo.infraestrutura;

import com.exemplo.dominio.Pedido;
import java.util.HashMap;
import java.util.Map;
import java.util.Optional;

public class PedidoRepositorio {

    private final Map<Long, Pedido> armazenamento = new HashMap<>();

    public void salvar(Pedido pedido) {
        armazenamento.put(pedido.getId(), pedido);
    }

    public Optional<Pedido> buscarPorId(Long id) {
        return Optional.ofNullable(armazenamento.get(id));
    }

}

```

```
java
// -----
// CAMADA DE APLICAÇÃO
// Orquestra o caso de uso sem conter regras de negócio
// -----
package com.exemplo.aplicacao;

import com.exemplo.dominio.Pedido;
import com.exemplo.infraestrutura.PedidoRepositorio;

public class PedidoServico {

    private final PedidoRepositorio repositorio;

    public PedidoServico(PedidoRepositorio repositorio) {
        this.repositorio = repositorio;
    }

    public Pedido criarPedido(Long id, String cliente) {
        Pedido pedido = new Pedido(id, cliente);
        repositorio.salvar(pedido);
        return pedido;
    }

    public void adicionarItem(Long pedidoId, double valor) {
        Pedido pedido = repositorio.buscarPorId(pedidoId)
            .orElseThrow(() -> new RuntimeException("Pedido não encontrado: " +
        pedido.adicionarItem(valor);
        repositorio.salvar(pedido);
    }

    public double consultarTotal(Long pedidoId) {
        return repositorio.buscarPorId(pedidoId)
            .map(Pedido::getTotal)
            .orElseThrow(() -> new RuntimeException("Pedido não encontrado: " +
    }

}

```

```
java
```

```
// -----
// CAMADA DE APRESENTAÇÃO
// Ponto de entrada – poderia ser um controller REST
// -----
package com.exemplo.apresentacao;

import com.exemplo.aplicacao.PedidoServico;
import com.exemplo.infraestrutura.PedidoRepositorio;

public class Main {

    public static void main(String[] args) {
        PedidoRepositorio repositorio = new PedidoRepositorio();
        PedidoServico servico = new PedidoServico(repositorio);

        servico.criarPedido(1L, "Ana Silva");
        servico.adicionarItem(1L, 49.90);
        servico.adicionarItem(1L, 12.50);

        double total = servico.consultarTotal(1L);
        System.out.println("Total do pedido: R$ " + total); // R$ 62.4
    }
}
```

Pontos de atenção: O domínio (`Pedido`) não importa nada de infraestrutura ou framework. A camada de serviço orquestra o fluxo, mas as regras de negócio (`adicionarItem`) residem na entidade. Isso facilita testes unitários de domínio sem qualquer mock.

Exemplo 2 — Arquitetura Hexagonal (Ports and Adapters)

A arquitetura hexagonal, proposta por Alistair Cockburn, posiciona o domínio no centro e isola completamente a lógica de negócio de qualquer detalhe externo. A comunicação com o exterior é feita por meio de **portas** (interfaces) e **adaptadores** (implementações concretas).

O exemplo modela um sistema de notificações:

```
java

// -----
// PORTA DE SAÍDA (interface definida pelo domínio)
// O domínio não sabe como a notificação será enviada
// -----
package com.exemplo.dominio.porta;

public interface ServicoNotificacao {
    void notificar(String destinatario, String mensagem);
}
```

```
java
```

```
// -----
// DOMÍNIO – Serviço de aplicação
// Depende apenas da abstração (porta)
// -----
package com.exemplo.dominio.servico;

import com.exemplo.dominio.porta.ServicoNotificacao;

public class ServicoUsuario {

    private final ServicoNotificacao notificacao;

    public ServicoUsuario(ServicoNotificacao notificacao) {
        this.notificacao = notificacao;
    }

    public void registrarUsuario(String email, String nome) {
        // Regra de negócio: validação básica
        if (email == null || !email.contains("@")) {
            throw new IllegalArgumentException("E-mail inválido.");
        }
        // ... persistência omitida para foco na arquitetura
        notificacao.notificar(email, "Bem-vindo, " + nome + "!");
    }
}
}
```

```
java
// -----
// ADAPTADOR DE E-MAIL (detalhe de infraestrutura)
// Implementa a porta definida pelo domínio
// -----
package com.exemplo.infraestrutura.adaptador;

import com.exemplo.dominio.porta.ServicoNotificacao;

public class AdaptadorEmail implements ServicoNotificacao {

    @Override
    public void notificar(String destinatario, String mensagem) {
        // Em produção: integração com JavaMail ou SendGrid
        System.out.println("[EMAIL] Para: " + destinatario + " | Mensagem: " + mensagem);
    }
}
}
```

```
java
// -----
// ADAPTADOR DE SMS (substituição transparente)
// O domínio não precisa ser alterado para trocar o canal
// -----
package com.exemplo.infraestrutura.adaptador;

import com.exemplo.dominio.porta.ServicoNotificacao;

public class AdaptadorSms implements ServicoNotificacao {

    @Override
    public void notificar(String destinatario, String mensagem) {
        System.out.println("[SMS] Para: " + destinatario + " | Mensagem: " + mensagem);
    }
}
}
```

```
java
```

```
// -----
// COMPOSIÇÃO (ponto de entrada ou container de DI)
// -----
package com.exemplo;

import com.exemplo.dominio.servico.ServicoUsuario;
import com.exemplo.infraestrutura.adaptador.AdaptadorEmail;
import com.exemplo.infraestrutura.adaptador.AdaptadorSms;

public class Main {

    public static void main(String[] args) {
        // Troca do adaptador sem alterar uma linha do domínio
        ServicoUsuario servicoComEmail = new ServicoUsuario(new AdaptadorEmail() {
            @Override public void registrarUsuario(String email, String nome) {
                servicoComEmail.registrarUsuario("joao@email.com", "João");
            }
        });

        ServicoUsuario servicoComSms = new ServicoUsuario(new AdaptadorSms() {
            @Override public void registrarUsuario(String email, String nome) {
                servicoComSms.registrarUsuario("maria@email.com", "Maria");
            }
        });
    }
}
```

Pontos de atenção: A inversão de dependência é explícita: `ServicoUsuario` (domínio) define a interface `ServicoNotificacao`; os adaptadores (`AdaptadorEmail`, `AdaptadorSms`) a implementam. Adicionar um novo canal de notificação (push, WhatsApp) exige apenas um novo adaptador — o domínio permanece intacto. Esse isolamento também permite substituir os adaptadores por **fakes** em testes, sem necessidade de Mockito.

Exemplo 3 — Microsserviços com Comunicação via Mensageria

Em sistemas de microsserviços, a comunicação assíncrona por mensageria é preferível à chamada síncrona direta entre serviços, pois aumenta o desacoplamento e a resiliência. O padrão **Publisher/Subscriber** é central nesse estilo.

O exemplo abaixo simula dois microsserviços independentes — `ServicoDeVendas` e `ServicoDeEstoque` — comunicando-se por um barramento de eventos (aqui simplificado como um `EventBus`) em memória para fins didáticos; em produção, usaria Apache Kafka ou RabbitMQ).

```
java
// -----
// CONTRATO DE EVENTO (compartilhado entre serviços)
// Em microsserviços reais, publicado como biblioteca
// -----
package com.exemplo.eventos;

public class PedidoRealizadoEvento {

    private final Long pedidoId;
    private final String produto;
    private final int quantidade;

    public PedidoRealizadoEvento(Long pedidoId, String produto, int quantidade) {
        this.pedidoId = pedidoId;
        this.produto = produto;
        this.quantidade = quantidade;
    }

    public Long getPedidoId() { return pedidoId; }
    public String getProduto() { return produto; }
    public int getQuantidade() { return quantidade; }
}
```

java

```
// -----  
// EVENT BUS SIMPLIFICADO  
// Substitui Kafka/RabbitMQ para fins de exemplo  
// -----  
package com.exemplo.infraestrutura;  
  
import com.exemplo.eventos.PedidoRealizadoEvento;  
import java.util.ArrayList;  
import java.util.List;  
import java.util.function.Consumer;  
  
public class EventBus {  
  
    private static final EventBus INSTANCIA = new EventBus();  
    private final List<Consumer<PedidoRealizadoEvento>> assinantes = new ArrayList<>();  
  
    private EventBus() {}  
  
    public static EventBus getInstance() {  
        return INSTANCIA;  
    }  
  
    public void assinar(Consumer<PedidoRealizadoEvento> handler) {  
        assinantes.add(handler);  
    }  
  
    public void publicar(PedidoRealizadoEvento evento) {  
        assinantes.forEach(handler -> handler.accept(evento));  
    }  
}
```

java

```
// -----  
// MICROSERVIÇO DE VENDAS (Producer)  
// Publica evento ao concluir a venda  
// -----  
package com.exemplo.vendas;  
  
import com.exemplo.eventos.PedidoRealizadoEvento;  
import com.exemplo.infraestrutura.EventBus;  
  
public class ServicoDeVendas {  
  
    private final EventBus eventBus;  
  
    public ServicoDeVendas(EventBus eventBus) {  
        this.eventBus = eventBus;  
    }  
  
    public void realizarPedido(Long pedidoId, String produto, int quantidade) {  
        // Lógica de negócio: validações, persistência, etc.  
        System.out.println("[Vendas] Pedido #" + pedidoId + " registrado: " +  
            quantidade + "x " + produto);  
  
        // Publica evento - não conhece quem irá consumi-lo  
        eventBus.publicar(new PedidoRealizadoEvento(pedidoId, produto, quantidade));  
    }  
}
```

java

```
// -----
// MICROSERVIÇO DE ESTOQUE (Consumer)
// Reage ao evento de pedido de forma independente
// -----
package com.exemplo.estoque;

import com.exemplo.eventos.PedidoRealizadoEvento;
import com.exemplo.infraestrutura.EventBus;
import java.util.HashMap;
import java.util.Map;

public class ServicoDeEstoque {

    private final Map<String, Integer> estoque = new HashMap<>();

    public ServicoDeEstoque(EventBus eventBus) {
        // Inicializa estoque fictício
        estoque.put("Notebook", 50);
        estoque.put("Mouse", 200);

        // Registra-se como consumidor do evento
        eventBus.assinar(this::aoReceberPedido);
    }

    private void aoReceberPedido(PedidoRealizadoEvento evento) {
        String produto = evento.getProduto();
        int quantidade = evento.getQuantidade();

        estoque.merge(produto, -quantidade, Integer::sum);

        System.out.println("[Estoque] Baixa realizada: " + quantidade + "x " +
            + " | Saldo atual: " + estoque.getOrDefault(produto, 0));
    }
}

```

```
java
// -----
// COMPOSIÇÃO DA APLICAÇÃO
// -----
package com.exemplo;

import com.exemplo.estoque.ServicoDeEstoque;
import com.exemplo.infraestrutura.EventBus;
import com.exemplo.vendas.ServicoDeVendas;

public class Main {

    public static void main(String[] args) {
        EventBus eventBus = EventBus.getInstance();

        // Os dois serviços só se conhecem pelo EventBus
        new ServicoDeEstoque(eventBus);
        ServicoDeVendas vendas = new ServicoDeVendas(eventBus);

        vendas.realizarPedido(101L, "Notebook", 2);
        vendas.realizarPedido(102L, "Mouse", 5);
    }
}

```

Saída esperada:

[Vendas] Pedido #101 registrado: 2x Notebook
[Estoque] Baixa realizada: 2x Notebook | Saldo atual: 48
[Vendas] Pedido #102 registrado: 5x Mouse
[Estoque] Baixa realizada: 5x Mouse | Saldo atual: 195

Pontos de atenção: `ServicoDeVendas` não importa nenhuma classe de `ServicoDeEstoque` — o acoplamento é zero. Novos consumidores (cobrança, logística, analytics) podem se inscrever no `EventBus` sem qualquer modificação no serviço de vendas, respeitando o **Open/Closed Principle**. Em produção, o `EventBus` seria substituído por uma integração com Apache Kafka, preservando a mesma estrutura lógica.

Quadro Comparativo

Critério	Arquitetura em Camadas	Hexagonal	Microserviços
Complexidade inicial	Baixa	Média	Alta
Testabilidade	Moderada	Alta	Alta (isolado por serviço)
Escalabilidade	Limitada	Moderada	Alta
Acoplamento	Por camada	Baixo (via porta)	Muito baixo (por evento)
Implantação	Única unidade	Única unidade	Múltiplos processos
Indicado para	Sistemas internos, MVPs	Domínios complexos	Alta escala, times grandes

Considerações Finais

A seleção de uma arquitetura não é decisão única e irreversível. Sistemas evoluem: é comum iniciar com uma arquitetura monolítica bem estruturada em camadas ou hexagonal, e migrar para microserviços à medida que a escala e a independência de times se tornam necessidades reais — estratégia conhecida como **Strangler Fig Pattern**.

O critério determinante não é a modernidade do estilo arquitetural, mas sim sua adequação ao contexto: tamanho da equipe, volume de transações, prazo de entrega e maturidade das práticas de engenharia envolvidas.